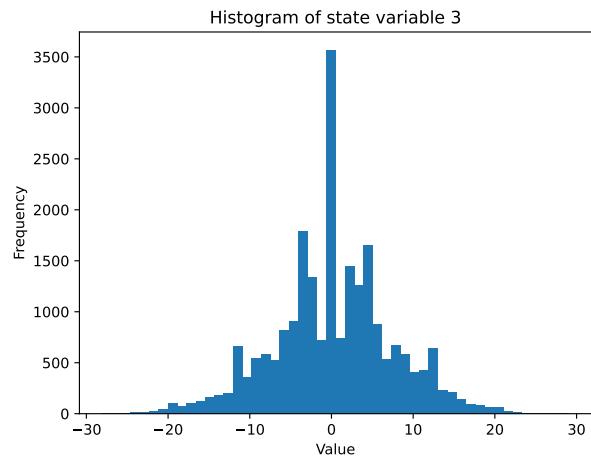
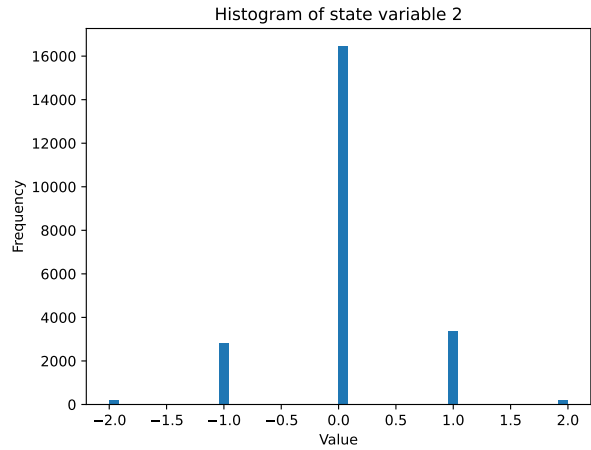
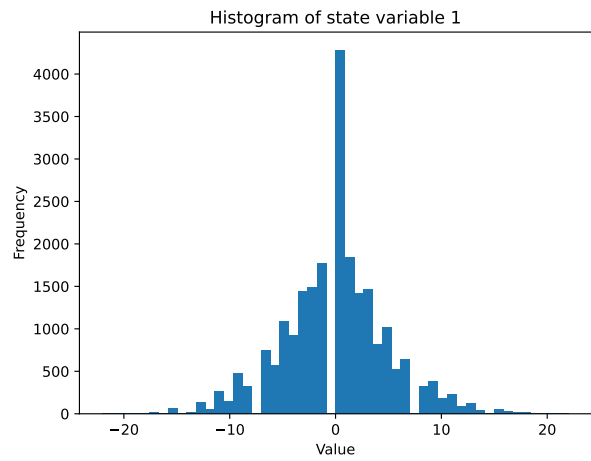
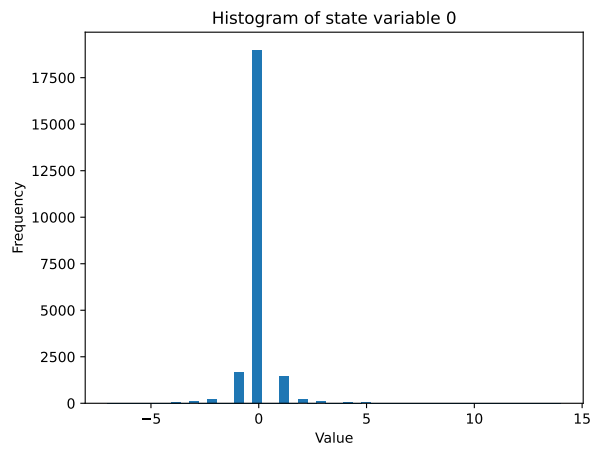


STA 5635 HW 12

Palmer Swanson

April 7, 2026

Part 1



State Variable	Min	Max	Num Discrete Values
0	-7	14	22
1	-22	22	45
2	-2	2	5
3	-28	29	58
Total			287100

Parts 2, 3, and 4

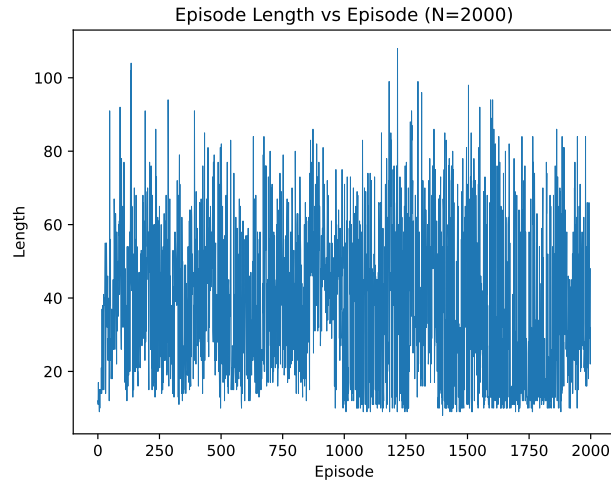


Figure 2: Episode Length vs Episode (N=2000)

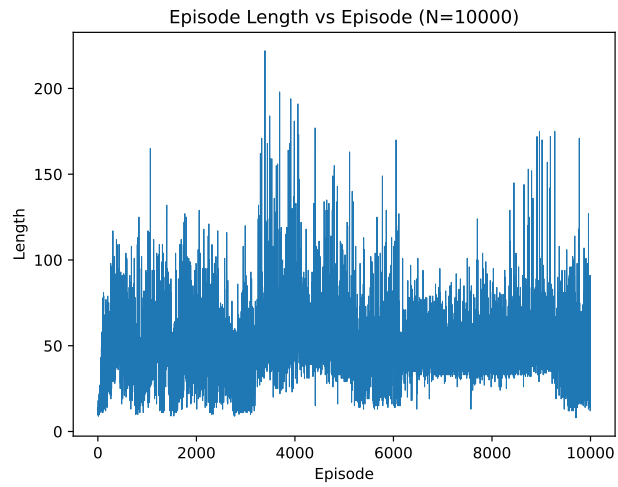


Figure 3: Episode Length vs Episode (N=10000)

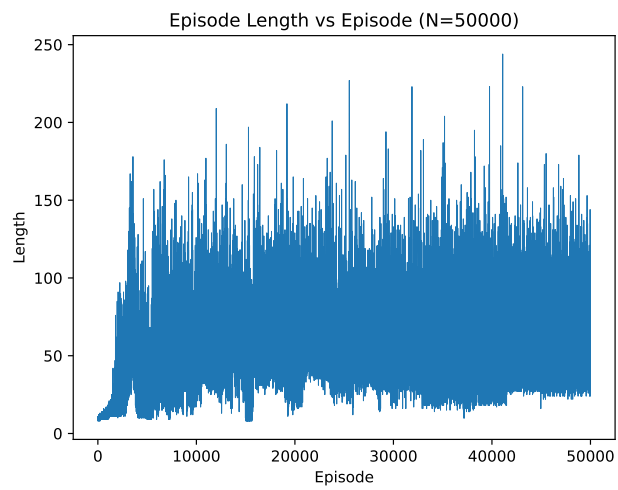


Figure 4: Episode Length vs Episode (N=50000)

N	% Nonzero (Random)	% Nonzero (Max)	Avg Ep. Length (Last 1000)
2000	0.724	1.003	37.348
10000	1.579	2.088	48.305
50000	3.187	4.147	68.446

Code

```
1 import gymnasium as gym
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import random
6
7 #need to create a cartpole environment
8 env = gym.make("CartPole-v1")
9
10 n_episodes = 1000
11
12 states = []
13
14 #part a
15 #loop over and collect the states of 1000 episodes
16 for i in range(n_episodes):
17     state, info = env.reset()
18
19     done = False
20
21     while not done:
22         #append to the states list
23         states.append(state)
24
25         a = random.randrange(2)
26         observation, reward, terminated, truncated, info = env.step(a)
27
28         done = terminated or truncated
29
30         #update state
31         state = observation
32
33 env.close()
34
35 #need to multiply by 10 and converting to integers
36 states = (np.array(states)*10).astype(int)
37
38 #plot histograms and save
39 #usig 50 bins for each state variable, and save as pdfs in the figures folder
40 for i in range(4):
41     plt.figure()
42     plt.hist(states[:, i], bins=50)
43     plt.title(f"Histogram of state variable {i}")
44     plt.xlabel("Value")
45     plt.ylabel("Frequency")
```

```

46     plt.savefig(f"assignments/hw12/figures/hist_state_variable_{i}.pdf",
47                 dpi=400, bbox_inches="tight")
48     plt.close()
49     #plt.show()
50
51     #need to find the range of each of the state variables
52     #also need to report the number of discrete sate values for each state variable
53     mins = states.min(axis=0)
54     maxs = states.max(axis=0)
55     num_vals = maxs - mins + 1
56
57     #get total number of discrete states
58     num_total_states = np.prod(num_vals)
59     print(f"Total number of discrete states: {num_total_states}")
60
61     #build dataframe
62     df = pd.DataFrame({
63         "State_Variable": [0, 1, 2, 3],
64         "Min": mins,
65         "Max": maxs,
66         "Num_Discrete_Values": num_vals
67     })
68
69     #add a total row
70     total_row = pd.DataFrame({
71         "State_Variable": ["Total"],
72         "Min": [""],
73         "Max": [""],
74         "Num_Discrete_Values": [num_total_states]
75     })
76
77     df = pd.concat([df, total_row], ignore_index=True)
78
79     latex_table = df.to_latex(index=False)
80
81     with open("assignments/hw12/output/state_summary_part1.tex", "w") as f:
82         f.write(latex_table)
83
84     #loop over these
85     N_values = [2000, 10000, 50000]
86     gamma = 0.9
87
88     results = []
89
90     for N in N_values:
91         print(f"\n=====N={N}=====")

```

```

92
93 #initialize Q table
94 Q = np.zeros((num_vals[0], num_vals[1], num_vals[2], num_vals[3], 2))
95
96 for i in range(N):
97     state, info = env.reset()
98     done = False
99
100     while not done:
101         #convert state to discrete state and clip to the min and max values
102         discrete_state = (state * 10).astype(int)
103         discrete_state = np.clip(discrete_state, mins, maxs)
104         s_idx = discrete_state - mins
105
106         #take random action
107         a = random.randrange(2)
108         next_state, reward, terminated, truncated, info = env.step(a)
109         done = terminated or truncated
110
111         #convert next state to discrete state and clip to the min and max
112         values
113         disc_next = (next_state * 10).astype(int)
114         disc_next = np.clip(disc_next, mins, maxs)
115         s_next_idx = disc_next - mins
116
117         #update Q table
118         Q[s_idx[0], s_idx[1], s_idx[2], s_idx[3], a] = (
119             reward + gamma * np.max(
120                 Q[s_next_idx[0], s_next_idx[1], s_next_idx[2], s_next_idx[3]]
121             )
122         )
123
124         state = next_state
125
126     #report percent nonzero after random phase
127     percent1 = 100 * np.count_nonzero(Q) / Q.size
128     print(f"Percent_non-zero_in_Q-table_with_random_actions:_{percent1:.2f}%")
129
130     eps = []
131
132     for i in range(N):
133         state, info = env.reset()
134         done = False
135         steps = 0
136
137         #stop any episode with more than 2000 steps
138         while not done and steps < 2000:

```

```

138     #convert state to discrete state
139     discrete_state = (state * 10).astype(int)
140     discrete_state = np.clip(discrete_state, mins, maxs)
141     s_idx = discrete_state - mins
142
143     a = np.argmax(Q[s_idx[0], s_idx[1], s_idx[2], s_idx[3]])
144
145     next_state, reward, terminated, truncated, info = env.step(a)
146     done = terminated or truncated
147
148     #convert next state to discrete state
149     disc_next = (next_state * 10).astype(int)
150     disc_next = np.clip(disc_next, mins, maxs)
151     s_next_idx = disc_next - mins
152
153     #update Q table
154     Q[s_idx[0], s_idx[1], s_idx[2], s_idx[3], a] = (
155         reward + gamma * np.max(
156             Q[s_next_idx[0], s_next_idx[1], s_next_idx[2], s_next_idx[3]]
157         )
158     )
159
160     #set state to next state and increment steps
161     state = next_state
162     steps += 1
163
164     #memorize episode length
165     eps.append(steps)
166
167     #report percent non zero after max action phase
168     percent2 = 100 * np.count_nonzero(Q) / Q.size
169     avg_last_1000 = np.mean(eps[-1000:])
170
171     print(f"Percent_non-zero_in_Q-table_after_max_value_action: {percent2:.2f}%")
172     print(f"Average_episode_length_(last_1000_episodes): {avg_last_1000:.2f}")
173
174     #save objects we need to report in a table
175     results.append({
176         "N": N,
177         "%_Nonzero_(Random)": percent1,
178         "%_Nonzero_(Max)": percent2,
179         "Avg_Ep._Length_(Last_1000)": avg_last_1000
180     })
181
182     # save plot
183     plt.figure()
184     plt.plot(eps, linewidth=0.3)

```

```

185     plt.title(f"Episode_Length_vs_Episode_(N={N})")
186     plt.xlabel("Episode")
187     plt.ylabel("Length")
188     plt.savefig(f"assignments/hw12/figures/episode_length_N{N}.pdf", dpi=300,
189               bbox_inches="tight")
190     plt.close()
191
192     #convert results to df and export
193     results_df = pd.DataFrame(results)
194
195     latex_table = results_df.to_latex(
196         index=False,
197         escape=True,
198         float_format="%.3f"
199     )
200
201     with open("assignments/hw12/output/results_summary.tex", "w") as f:
202         f.write(latex_table)

```